Laboratory 0b101

Expected submission ON PORTALE DELLA DIDATTICA: SURNAME_NAME_MATRICOLA_lab_05.zip

Including (in case you have projects with the same name, you can prepend lab_5 to them):
- program_1
- lab_05.pdf (complete this file and export it as PDF)

Submission deadline: 03/11/2026

Exercise 1:

1) Write a new assembly program (program_1) that implements the following C code:

```
for (i = 0; i < 100; i++) {
      v4[i] = (v1[i] / v2[i]) + v3[i];
      if (int(i/3) == 0) {
            v5[i] = v4[i] * v3[i];

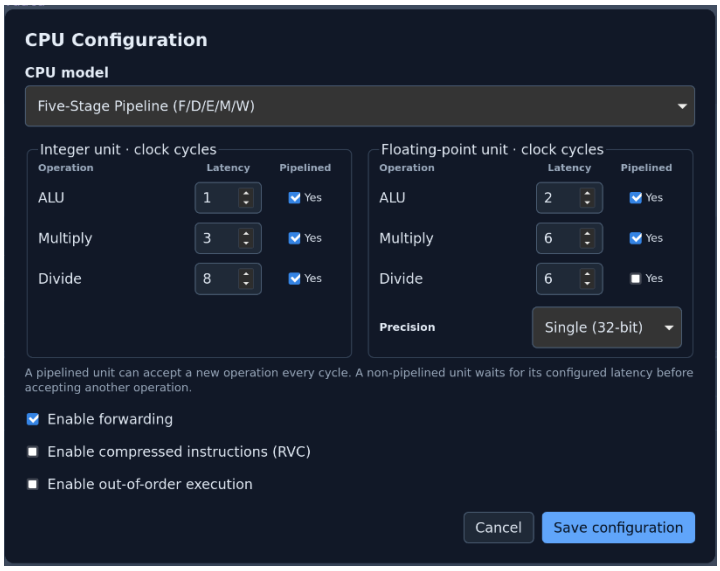
      } else if (int(i/3) == 1) {
            v5[i] = v4[i] / v3[i];

      } else {
            v5[i] = v4[i] + v3[i];
      }
}
```

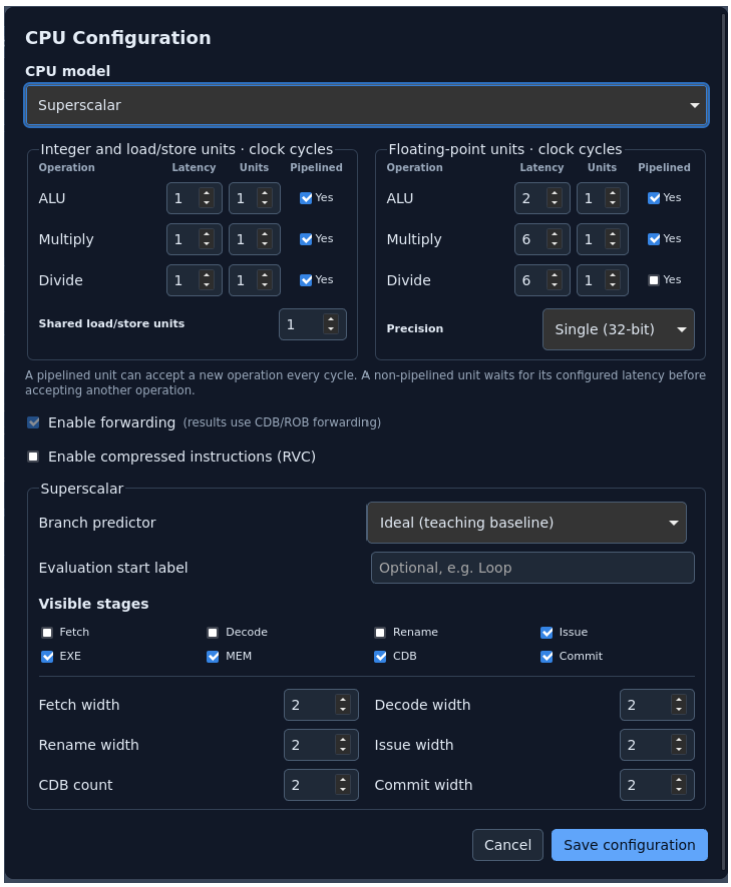
Considering v2 and v3 as arrays of 100 floating point values, always different from zero

2) Simulate the program using the two CPU configurations shown below. Configuration 1 uses an In-Order CPU, while Configuration 2 uses an Superscalar CPU. In the second case the Ideal BPU is active (i.e., assume that every branch prediction is correct).

Configuration 1)



Configuration 2)



Run both configurations and record the resulting CPI (Clocks Per Instruction) for the In-Order and the Superscalar CPU in the table below:

	program_1
CPI In-Order CPU	
CPI Superscalar CPU	

Compare the two results: which architecture performs better, and why does this happen? Justify your answer.

Answer:

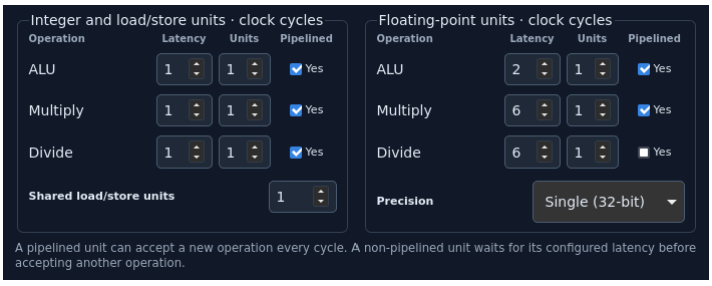
By hand, trace the first three iterations of the loop on the Superscalar CPU and fill in the table below, writing the clock cycle at which each instruction passes through every pipeline stage (Issue, EXE, MEM, CDB, COMMIT):

# iteration		Issue	EXE	MEM	CDB x2	COMMIT x2
1						
1						
1						
1						
1						
1						
1						
1						

1						
1						
1						
2						
2						
2						
2						
2						
2						
2						
2						
2						
2						
2						
2						
2						
3						
3						
3						
3						
3						
3						
3						
3						
3						
3						
3						
3						
3						

Exercise 2: Architectural Changes

In this exercise you will study how the amount of execution resources affects performance. Starting again from the Superscalar configuration of Exercise 1 (with the Ideal BPU), change the number of Integer Load/Store Units (LSU) from 1 to 2. For each configuration, record the CPI in a table and explain what happens: identify which resource becomes the bottleneck, and describe how adding LSUs changes the number of structural stalls and therefore the overall performance. Comment on whether increasing the number of units improves the CPI.



	program_1
Initial Configuration	
CPI 2 LSUs	

Answer:

(OPTIONAL) Exercise 3: Branch Prediction Unit (BPU)

Using the Superscalar configuration from Exercise 1, study how the Branch Prediction Unit (BPU) affects performance. Run program_1 first with the Ideal BPU (the teaching baseline, in which every prediction is assumed correct) and then with the Local Counter BPU.

Report for both the CPI and explain why they differ, if so.

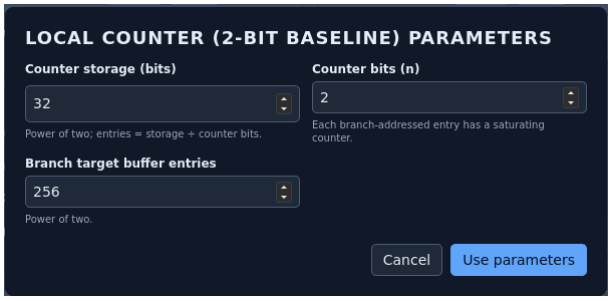
	program_1
CPI Ideal BPU	
CPI Local Counter BPU	

Answer:

Starting from the Local Counter BPU, perform the following two experiments, changing one parameter at a time while keeping the other fixed:

a) vary the number of counter bits from 2 to 4 while keeping the size of the prediction table constant.

b) vary the number of counters in the history table from 2 to 8 while keeping the number of counter bits constant. For each setting, record the CPI of the Superscalar CPU.



For every configuration, report the CPI in a table and explain what changes during execution: how the prediction accuracy affects the number of mispredictions, and therefore the pipeline flushes and stalls, and how this is reflected in the CPI. Finally, compare all your results against the Ideal BPU baseline and comment on how close the Local Counter BPU can get to it.

	program_1
Initial Configuration	
CPI Counter Bits 2-> 8	
CPI Counter storage 32 -> 64	

Answer: